

Intraday CFDs Trading Agent Using Deep Reinforcement Learning

Chayanin Napia 6509
Pronpipath Neamnoi 6571

April 19, 2026

1 Introduction

Recent advances in deep reinforcement learning (DRL) have enabled trading to be modeled as a sequential decision-making problem, making it a natural fit for Contracts for Difference (CFDs), derivative instruments that allow speculation on price movements without owning the underlying asset. In intraday CFD trading, an agent must decide when to enter, hold, or close a position under constraints such as transaction costs, spread, and leverage limits.

This project applies DRL to XAUUSD (spot gold priced in US dollars), selected for its deep liquidity, high intraday volatility driven by macroeconomic events, and recurring patterns tied to the London and New York sessions. A realistic intraday CFD environment is designed and agent performance is evaluated under backtesting conditions.

2 Objectives and Scopes

Objectives:

- To construct an intraday DRL trading environment for XAUUSD under realistic same-day trading constraints.
- To design the state space using OHLCV-derived price features together with positional features.
- To compare the performance of Double DQN, A2C, and PPO under the same trading environment.
- To evaluate the trained agents in terms of profitability, stability, and overall intraday trading performance under transaction costs.

Scopes:

- XAUUSD CFD on MetaTrader 5 (MT5) using broker-provided historical data; historical simulation and backtesting only.
- Intraday trading only; no overnight position holding; 1-minute timeframe.
- Observation space limited to OHLCV-derived price and positional features from MT5; external information (news, macroeconomic data, order book) is excluded.

3 Literature Review

Zhang et al. [1] show that DRL can learn profitable position-taking policies under transaction costs across multiple asset classes, demonstrating that market states can be mapped directly to trade decisions without separating prediction from execution.

Goluzza et al. [2] demonstrate that price-based features alone are insufficient for intraday trading; adding positional context such as unrealized return, daily return, and time remaining in the session improves agent performance. This motivates the inclusion of positional features in the observation space of this project.

DeepScalper [3] further extends intraday DRL with a risk-aware auxiliary objective using limit order book data, though its complexity exceeds a practical retail CFD setting. Collectively, these works confirm that DRL can optimise under transaction costs, positional context is critical intraday, and OHLCV-based environments remain a valid research scope.

4 Problem Formulation

4.1 Markov Decision Process Formalisation

The intraday gold trading task is formulated as a Markov Decision Process (MDP), represented by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. At each decision step, the agent observes the current market and portfolio condition as a state $s_t \in \mathcal{S}$, chooses a trading action $a_t \in \mathcal{A}$, and transitions to the next state s_{t+1} according to the market dynamics and portfolio update rule. The reward $r_t = R(s_t, a_t, s_{t+1})$ reflects the quality of the action in terms of profitability and risk-aware trading behavior. The discount factor $\gamma \in [0, 1)$ is used to balance short-term gains with long-term cumulative performance.

In this formulation, one episode corresponds to a defined intraday trading horizon over historical XAUUSD data. The goal of the agent is to learn a policy that maximizes expected cumulative reward while operating under realistic constraints such as transaction costs, position changes, and noisy market conditions.

4.2 State Space

The state space is defined from two feature groups selected for this project: price-based features and positional features. The reference paper is used only as motivation for including positional context, while the feature set itself follows the design choices of this proposal.

Price-Based Features

Close returns over lookbacks $w \in \{1, 5, 15, 30, 60\}$ bars capture multi-horizon momentum and short-term directional movement:

$$r_{t-w,t} = \frac{p_t^C - p_{t-w}^C}{p_{t-w}^C}.$$

MACD captures directional structure and trend continuation or weakening, computed from the closing price as

$$\text{MACD}_t = \text{EMA}_{12}(C_t) - \text{EMA}_{26}(C_t),$$

where EMA_{12} and EMA_{26} denote the 12- and 26-period exponential moving averages.

Stochastic Oscillator and **RSI** capture short-term momentum, buying and selling pressure, and possible reversal zones. The Stochastic Oscillator over lookback n is

$$\text{STO}_t = \frac{C_t - L_t^{(n)}}{H_t^{(n)} - L_t^{(n)}} \times 100,$$

where $H_t^{(n)}$ and $L_t^{(n)}$ are the highest high and lowest low over the last n bars. RSI is computed as

$$RS_t = \frac{\bar{G}_t}{\bar{L}_t}, \quad RSI_t = 100 - \frac{100}{1 + RS_t},$$

where $\bar{G}_t$ and $\bar{L}_t$ are the average gain and average loss over n bars.

ATR represents the current volatility regime. The true range is first computed as

$$TR_t = \max(H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|),$$

and the Average True Range over n bars is $ATR_t = \frac{1}{n} \sum_{i=0}^{n-1} TR_{t-i}$.

Spread is included to represent the current execution cost faced by the agent:

$$SP_t = \text{Ask}_t - \text{Bid}_t.$$

SP_t is obtained from the broker-provided MT5 data feed and is included because transaction cost is modeled from spread.

Positional Features

Market signals alone cannot tell the agent whether it is already exposed, how long it has been holding, how far the position is from break-even, or how close the session end is. Five positional features are therefore added to the state.

Table 1: Positional features (p_t)

Symbol	Name	Definition
TL_t	Time Left	$T - t$ (bars until forced close)
POS_t	Position	$\in \{-1, 0, 1\}$ (short, flat, long)
PR_t	Unrealized Return	$\frac{POS_{t_{enter}} \cdot (C_t - O_{t_{enter}+1}) - tx_{t_{enter}+1}}{O_{t_{enter}+1}}$
DR_t	Daily Return	$\sum_{i=1}^t r_i$
HT_t	Holding Time	$t - t_{enter}$ in bars (0 if flat)

The full state vector combining both groups is

$$s_t = [m_t, p_t] \in \mathbb{R}^{15},$$

where $m_t = [R_t^{OHLCV}, \text{MACD}_t, \text{STO}_t, \text{ATR}_t, \text{RSI}_t, SP_t]$ and $p_t = [TL_t, POS_t, PR_t, DR_t, HT_t]$.

4.3 Action Space

The proposed action space is defined as a target-position action space:

$$a_t \in \{-1, 0, 1\},$$

where $a_t = -1$ is short, $a_t = 0$ is flat, and $a_t = 1$ is long. A fixed position size is used throughout, so the action determines only target direction. A reversal (e.g. $a_{t-1} = 1 \rightarrow a_t = -1$) incurs a larger transaction cost than maintaining or closing a position, making turnover explicit.

4.4 Reward Function

Following the reference paper, the reward is defined as the logarithmic return earned over each step after transaction cost:

$$r_{t+1} = \log \left(\frac{p_{exec} + a_t(p_{t+1}^C - p_{exec}) - tx_{t+1}}{p_{exec}} \right).$$

The transaction cost term is defined as

$$tx_{t+1} = COM \cdot p_{t+1}^O \cdot |a_t - a_{t-1}|,$$

and the effective execution price is

$$p_{exec} = \begin{cases} p_{t+1}^O, & \text{if } a_t \neq a_{t-1} \\ p_t^C, & \text{otherwise.} \end{cases}$$

Log-returns are additive across steps and align naturally with cumulative return maximisation, while the transaction cost term discourages unnecessary turnover.

4.5 Environment

The environment is designed as a fixed intraday trading session over historical XAUUSD data. One episode corresponds to one trading day, and one step corresponds to one **1-minute** bar. The trading session is fixed at 09:00--00:00 UTC+7 for all episodes. No overnight holding is allowed, and any open position is forcibly closed before the session ends.

Let T denote the number of **1-minute** bars in one trading session. The episode terminates after forced flattening at $t = T$. This fixed-session design isolates intraday behaviour, avoids overnight gap risk, and keeps the experimental setting consistent with a realistic same-day trading workflow.

4.6 Learning Algorithms

Three DRL algorithms are evaluated under the same MDP, targeting $\pi^* = \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} [\sum_{t=0}^T \gamma^t r_t]$.

Double DQN reduces overestimation bias by decoupling the target network. The TD target and loss are:

$$y_t = r_{t+1} + \gamma Q_{\theta^-} \left(s_{t+1}, \arg \max_a Q_{\theta}(s_{t+1}, a) \right), \quad \mathcal{L}_{DDQN}(\theta) = \mathbb{E} \left[(Q_{\theta}(s_t, a_t) - y_t)^2 \right].$$

A2C jointly learns policy π_{θ} and value V_w using advantage $\hat{A}_t = r_{t+1} + \gamma V_w(s_{t+1}) - V_w(s_t)$:

$$\mathcal{J}_{A2C}(\theta) = \mathbb{E} \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right], \quad \mathcal{L}_V(w) = \hat{A}_t^2.$$

PPO stabilizes updates via a clipped surrogate. With ratio $\rho_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$:

$$\mathcal{L}_{clip}(\theta) = \mathbb{E} \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad \mathcal{L}_{VF}(\theta) = \left(V_\theta(s_t) - \hat{V}_t \right)^2.$$

The PPO update maximizes a combined objective of $\mathcal{L}_{clip}$ and $\mathcal{L}_{VF}$, making it well suited to noisy financial trajectories.

5 System Overview

The proposed system consists of five main stages. First, historical XAUUSD data are retrieved as **1-minute OHLCV** bars. Second, market and positional features are engineered from the processed OHLCV series. Third, the data are filtered into a fixed daily session and wrapped into a custom intraday trading environment with transaction costs and forced end-of-session liquidation. Fourth, DRL agents are trained on this environment under the same problem setting. Finally, the trained agents are evaluated through backtesting and comparative performance analysis. Figure 1 illustrates the overall pipeline.

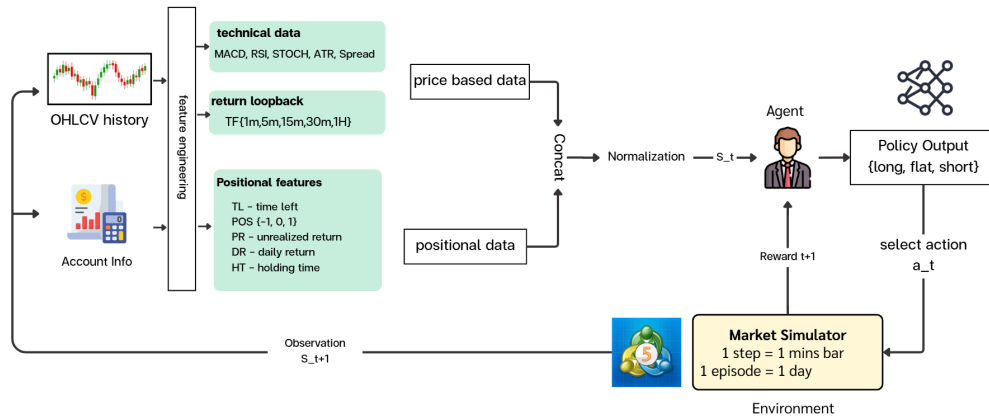


Figure 1: System diagram of the proposed intraday DRL trading framework.

Table 2 summarises the main frameworks and tools used in the system.

Table 2: Frameworks and tools

Tool / Framework	Role
MetaTrader 5 (MT5)	Broker-side historical market data source
MetaTrader5 Python package	Retrieve OHLCV data from MT5
Python	Main development environment
Pandas / NumPy	Data preprocessing and numerical computation
Technical indicator libraries	Derive MACD, Stochastic Oscillator, ATR, RSI
Gymnasium	Custom trading environment implementation
Stable-Baselines3	Double DQN, A2C, and PPO training
PyTorch	Deep learning backend
Backtesting modules	Performance evaluation and analysis

In the current scope, MT5 is used as a data source rather than a live execution platform. The agent interacts with the custom trading environment during training and backtesting, and trade actions are simulated inside the environment instead of being sent as real orders to MT5.

6 Performance Evaluation Metrics

Let $\{r_1, \dots, r_T\}$ be the daily return sequence over the backtesting period and $\bar{r} = \frac{1}{T} \sum_{t=1}^T r_t$ the mean daily return.

6.1 Sharpe Ratio

The Sharpe Ratio measures excess return per unit of total return volatility and serves as the primary metric for comparing the three algorithms:

$$\text{Sharpe} = \frac{\bar{r}}{\sigma(r)} \times \sqrt{252},$$

where $\sigma(r) = \sqrt{\frac{1}{T} \sum_{t=1}^T (r_t - \bar{r})^2}$ is the standard deviation of daily returns and $\sqrt{252}$ annualises the ratio.

6.2 Maximum Drawdown

Maximum Drawdown (MDD) measures the largest peak-to-trough decline in cumulative return during backtesting, capturing worst-case loss that the Sharpe Ratio alone does not reveal:

$$\text{MDD} = \max_{0 \leq i \leq j \leq T} \left(\frac{V_i - V_j}{V_i} \right),$$

where $V_t = \prod_{k=1}^t (1 + r_k)$ is the cumulative portfolio value at day t , with $V_0 = 1$.

6.3 Sortino Ratio

The Sortino Ratio penalises only downside deviation, leaving upside volatility unpenalised. Let $r_t^- = \min(r_t, 0)$ and downside deviation $\text{DD} = \sqrt{\frac{1}{T} \sum_{t=1}^T (r_t^-)^2}$. The annualised Sortino Ratio is:

$$\text{Sortino} = \frac{\bar{r}}{\text{DD}} \times \sqrt{252}.$$

References

- [1] Zhang, Z., Zohren, S., & Roberts, S. (2019). *Deep reinforcement learning for trading*. arXiv. <https://arxiv.org/abs/1911.10107>
- [2] Goluz, S., Kovacevic, T., Bauman, T., & Kostanjcar, Z. (2024). *Deep reinforcement learning with positional context for intraday trading*. Soft Computing. <https://doi.org/10.1007/s00500-024-09801-9>
- [3] Sun, S., Xue, W., Wang, R., He, X., Zhu, J., Li, J., & An, B. (2022). *DeepScalper: A risk-aware reinforcement learning framework to capture fleeting intraday trading opportunities*. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management* (pp. 1858–1867). Association for Computing Machinery. <https://doi.org/10.1145/3511808.3557429>